Punto 5 - Aplicación de patrones

Principios SOLID Aplicados

1. Principio de Responsabilidad Única (SRP - Single Responsibility Principle)

- **Problema detectado:** Varias clases del dominio estaban sobrecargadas con responsabilidades ajenas a su propósito principal. Agenda mezclaba la asignación de turnos con la gestión de urgencias, reportes y la escritura directa de notificaciones en archivos. Inventario mezclaba control de stock con reportes y alertas físicas. Además, Clinica gestionaba el personal y los clientes, pero también se encargaba de su propia persistencia (leer y guardar archivos).
- **Decisión tomada:** Se delegaron las responsabilidades adicionales a clases específicas, purificando las clases de dominio.
    - Las urgencias pasaron a GestorUrgencias y los reportes a GeneradorReportes.
    - Las notificaciones y alertas dejaron de ser responsabilidad de Agenda e Inventario para delegarse a la interfaz Notificador (y NotificadorLog).
    - La persistencia se extrajo de Clinica hacia la clase genérica Repositorio<T> y su implementación concreta RepositorioArchivo<T>.
- **Clases en las que se ve reflejado:** Agenda, Inventario, Clinica, GestorUrgencias, GeneradorReportes, la jerarquía de Repositorio y Notificador.
- **Ventaja:** Alta cohesión y bajo acoplamiento. Cada clase cambia ahora por un único motivo. Un cambio en la forma de guardar datos (persistencia) o de enviar alertas no requiere tocar la lógica de negocio de la Clínica, la Agenda o el Inventario.

2. Principio Abierto/Cerrado (OCP - Open/Closed Principle)

- **Problema detectado:** El método Factura.calcularTotal() dependía de múltiples sentencias condicionales (if/switch) basadas en el MedioPago (que era un *enum*). Si en el futuro se quería agregar un nuevo medio de pago (ej. billeteras virtuales), era obligatorio modificar el código interno de la clase Factura.
- **Decisión tomada:** Se transformó MedioPago en una interfaz con implementaciones concretas (Efectivo, Debito, Credito), encapsulando en cada una sus propias reglas de negocio y recargos (RN-F2, RN-F3, S-19).
- **Clases en las que se ve reflejado:** Factura, Interfaz MedioPago, Efectivo, Debito, Credito.
- **Ventaja:** El diseño queda abierto a la extensión y cerrado a la modificación. Se pueden incorporar nuevos medios de pago simplemente creando una nueva clase que implemente la interfaz, sin alterar la clase Factura.

3. Principio de Inversión de Dependencias (DIP - Dependency Inversion Principle)

- **Problema detectado:** Las clases de dominio como Agenda (en notificarDuenio) e Inventario (en notificarAdministrador) escribían directamente en un archivo de log físico (dependiendo

de un detalle de infraestructura mediante el atributo rutaLog). Cambiar el canal de notificación (por ejemplo, a un correo electrónico real) obligaba a modificar el núcleo del negocio.

- **Decisión tomada:** Se extrajo la lógica de notificación hacia una abstracción. Las clases de negocio ahora dependen de la interfaz Notificador. La escritura en el log se encapsuló en la implementación concreta NotificadorLog.
- **Clases en las que se ve reflejado:** ControladorTurnos, ControladorAtencion, GestorUrgencias, ServicioRecordatorios (quienes ahora delegan), Interfaz Notificador y clase NotificadorLog.
- **Ventaja:** El dominio se desacopla de los detalles de infraestructura (salida/I-O). Permite cambiar el mecanismo de notificación en un solo lugar y facilita el testing (se pueden usar *Mocks* de la interfaz sin escribir archivos reales).

Patrones GRASP Aplicados

1. Controlador

- **Problema detectado:** Los casos de uso carecían de un punto de entrada claro desde la interfaz de usuario. Si la capa de presentación se comunicaba directamente con las clases de dominio, esta tendría que coordinar la lógica, generando un fuerte acoplamiento.
- **Decisión tomada:** Se crearon cinco controladores lógicos, agrupados por familia de casos de uso, que actúan como intermediarios. Ellos reciben los eventos del sistema, coordinan las tareas y delegan el trabajo a los objetos de dominio correspondientes.
- **Clases en las que se ve reflejado:** ControladorRegistro, ControladorTurnos, ControladorAtencion, ControladorFacturacion, ControladorAdministracion.
- **Ventaja:** Mantiene la capa de interfaz gráfica completamente desacoplada de la lógica de negocio, centralizando la coordinación del sistema.

2. Experto en Información

- **Problema detectado:** Existía una mala distribución de la información. Por ejemplo, Recepcionista.emitirNotaCredito construía la nota usando importes de facturas ajenas. A su vez, Factura.calcularTotal debía recibir el "plan de salud" como parámetro porque desconocía a su cliente, forzando a quien la llamaba a navegar por toda la cadena de relaciones (Consulta -> Turno -> Mascota -> Duenio).
- **Decisión tomada:** Se asignó la responsabilidad a la clase que posee la información necesaria. Factura ahora genera la NotaCredito (método anular) con su propio importe total. Además, se asoció directamente Factura con Duenio, permitiéndole a la factura preguntarle directamente a su cliente si posee plan de salud.
- **Clases en las que se ve reflejado:** Factura, NotaCredito, Duenio, Recepcionista.
- **Ventaja:** Se reduce el acoplamiento al evitar largas cadenas de navegación y los cálculos se encapsulan en las clases expertas que poseen los datos.

3. Creador

- **Problema detectado:** Las partes de las relaciones de composición llegaban ya instanciadas desde el exterior (ej. HistoriaClinica.agregarConsulta(consulta), Factura.agregarLinea(linea)). Esto permitía la existencia de "partes" huérfanas sin su "todo", repartía la lógica de creación en los controladores y dificultaba un punto central para las validaciones.
- **Decisión tomada:** Se aplicó el patrón Creador garantizando que el "todo" instancie a sus "partes". Ahora, los métodos reciben parámetros primitivos o de datos y ejecutan el new internamente (HistoriaClinica crea la Consulta; Consulta crea el Tratamiento, etc.).
- **Clases en las que se ve reflejado:** HistoriaClinica, Consulta, Tratamiento, VacunaAplicada, NotaRectificatoria, Factura, LineaFactura.
- **Ventaja:** Se garantiza la integridad de la composición (las partes no existen sin su todo) y asegura que las reglas de negocio (ej. validar el peso antes de un tratamiento) se ejecuten en el momento exacto de la creación.

4. Fabricación Pura

- **Problema detectado:** Clases centrales del dominio estaban sobrecargadas con lógicas que no representaban conceptos puros del negocio real, disminuyendo su cohesión. Clinica mezclaba la gestión del personal con persistencia física (guardarEnArchivo), y Agenda mezclaba los turnos con notificaciones y reportes.
- **Decisión tomada:** Se crearon clases artificiales (Fabricaciones Puras) para agrupar estos comportamientos técnicos o de coordinación que no encajan en las abstracciones del mundo real.
- **Clases en las que se ve reflejado:** GestorUrgencias, GeneradorReportes, ServicioRecordatorios, NotificadorLog y Repositorio.
- **Ventaja:** Preserva la alta cohesión y el bajo acoplamiento en las clases del modelo de dominio, separando los conceptos del negocio de las utilidades técnicas o coordinaciones complejas.